TECNOLOGICO NACIONAL DE MEXICO

INSTITUTO TECNOLOGICO DE TUXTEPEC

INGENIERÍA EN SISTEMAS COMPUTACIONALES

ASIGNATURA:

CIENCIA DE DATOS

DOCENTE:

DR. JULIO AGUILAR CARMONA

NOMBRE DEL TRABAJO:

REPORTE “CURSO:AGENTES DE IA CON GOOGLE”

SEMESTRE: GRUPO:

8° **”A”**

NOMBRE DEL ALUMNO:

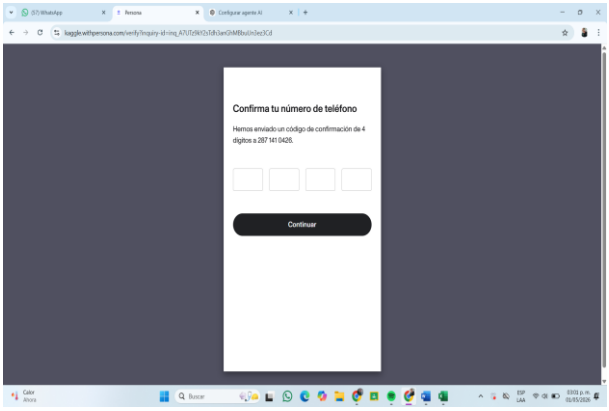
HIDALGO BAJANDO AURORA GUADALUPE

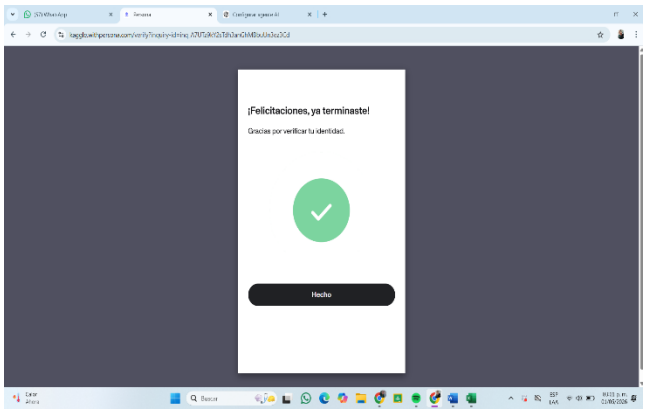
22350711

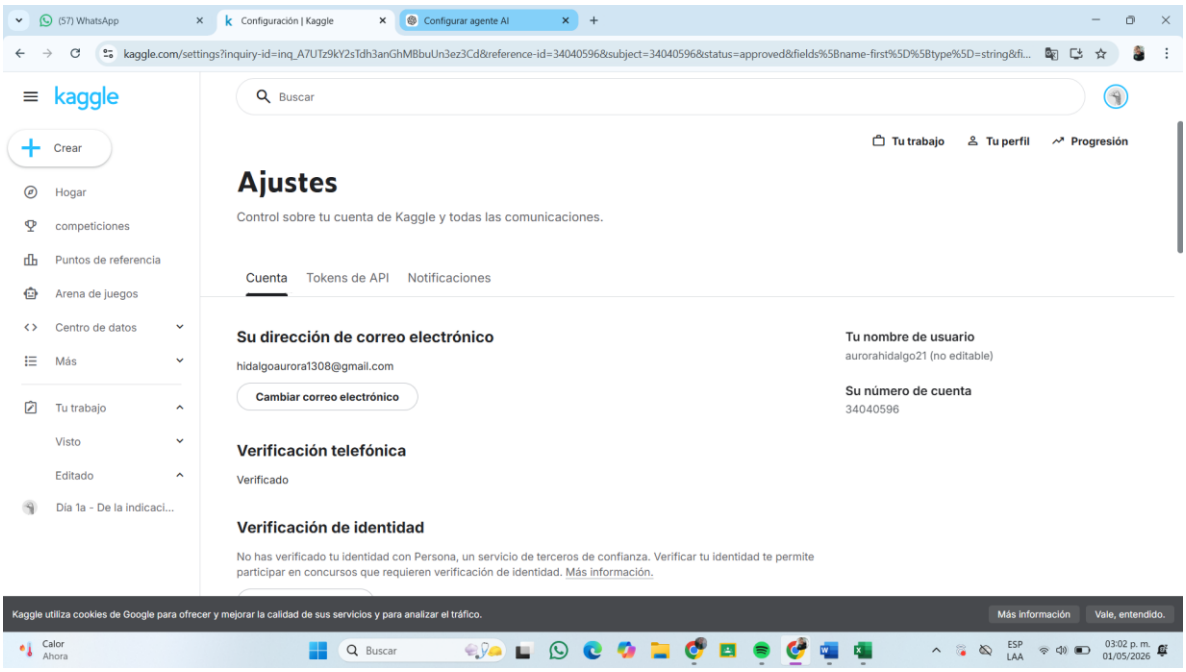
LUNES 27 DE ABRIL DE 2026.

CURSO INTENSIVO: “AGENTES DE IA CON GOOGLE”

Primeramente debemos verificar nuestra cuenta de kaggle con nuestro numero de teléfono para poder dar inicio al curso, y creamos nuestra GOOGLE_API_KEY personal para poder ejecutar los códigos

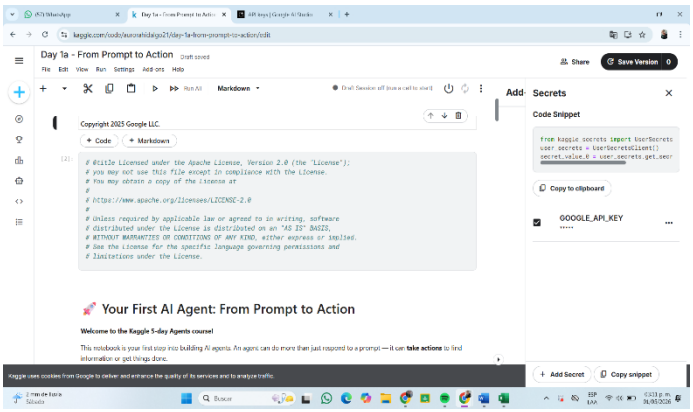


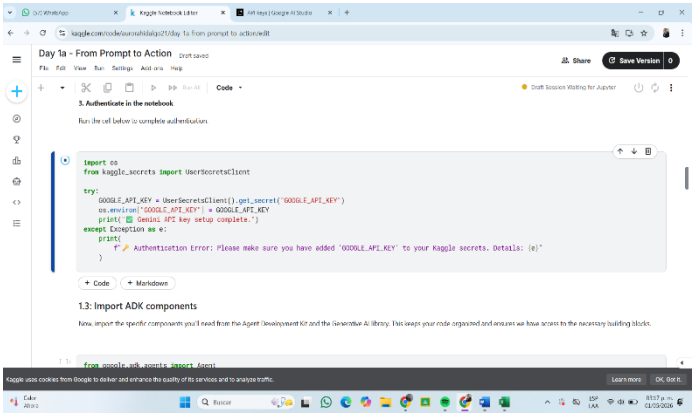


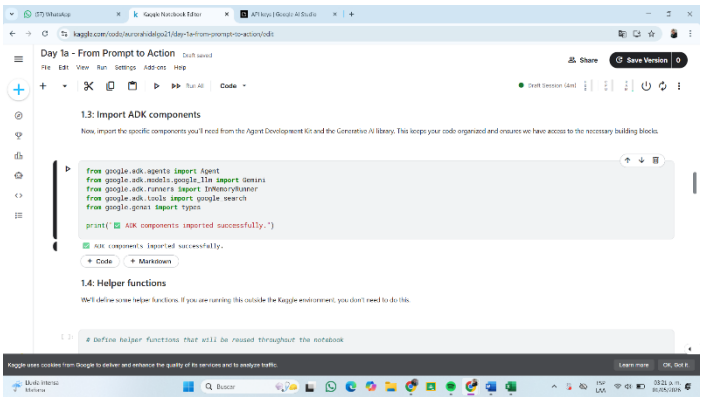


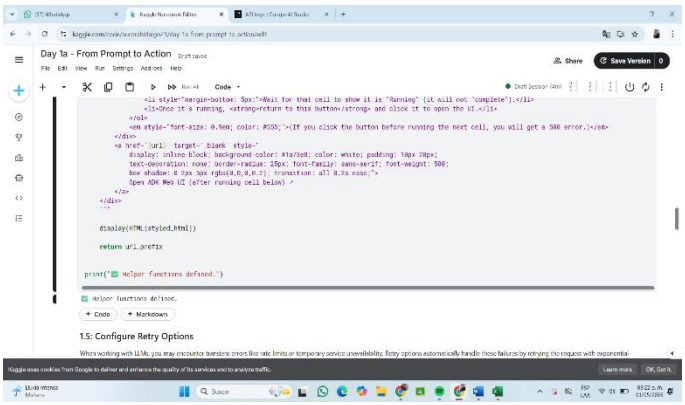
DIA 1 INTRODUCCIÓN AL DESARROLLO DE AGENTES CON ADK

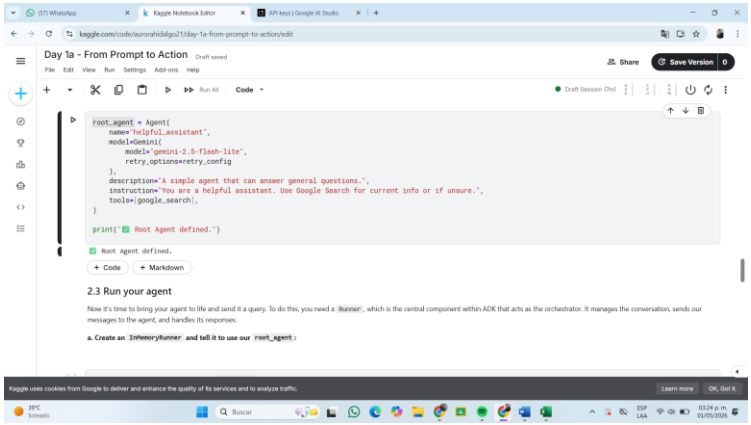
Configuramos el entorno, depues empezamos a Importar librerías de ADK. Luego Creamos un agente básico.

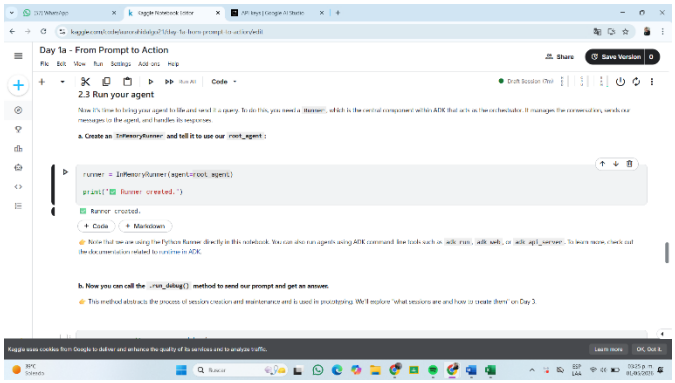


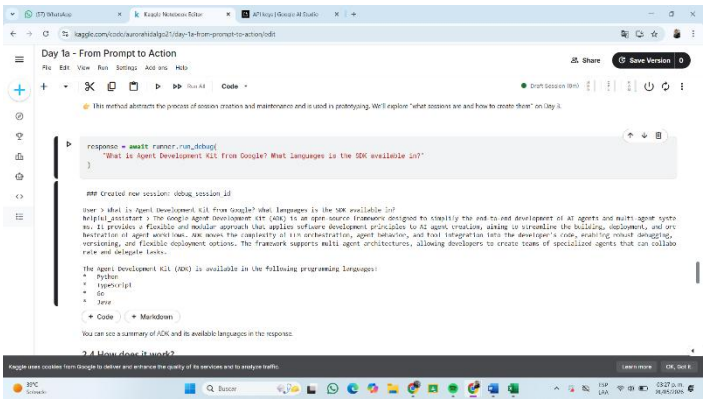


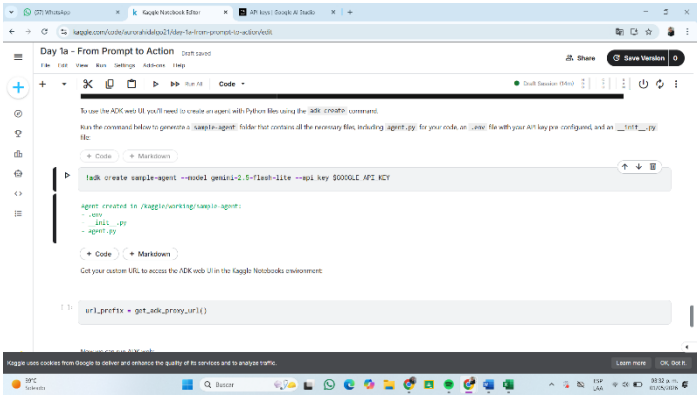




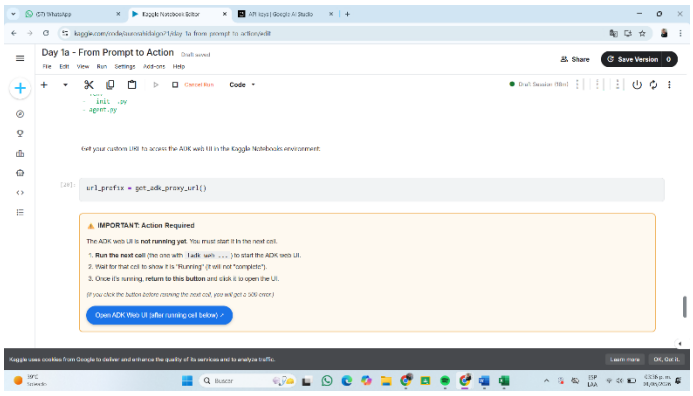


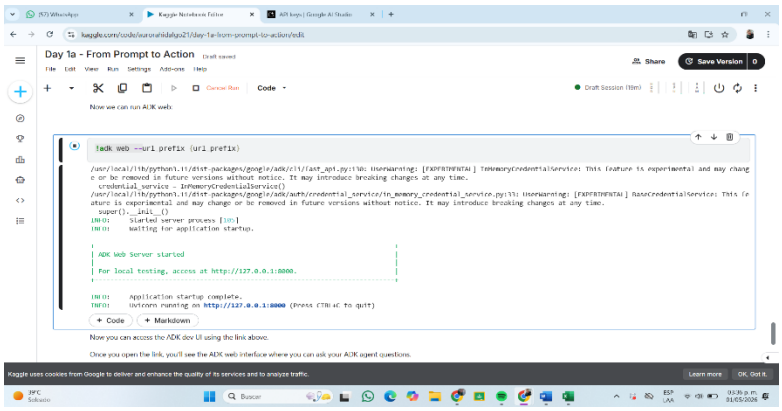


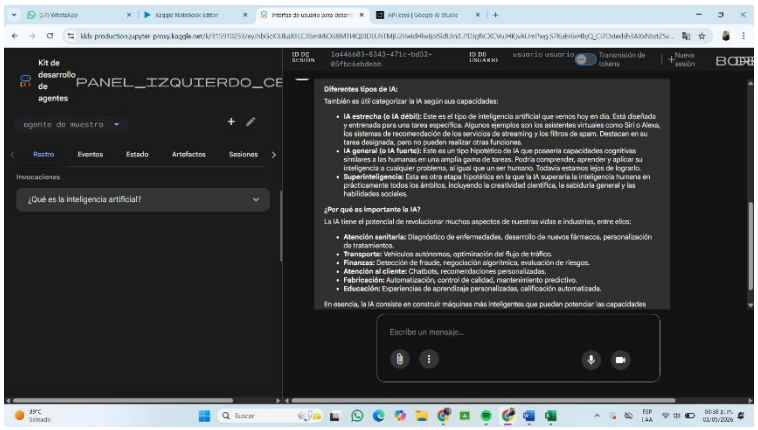


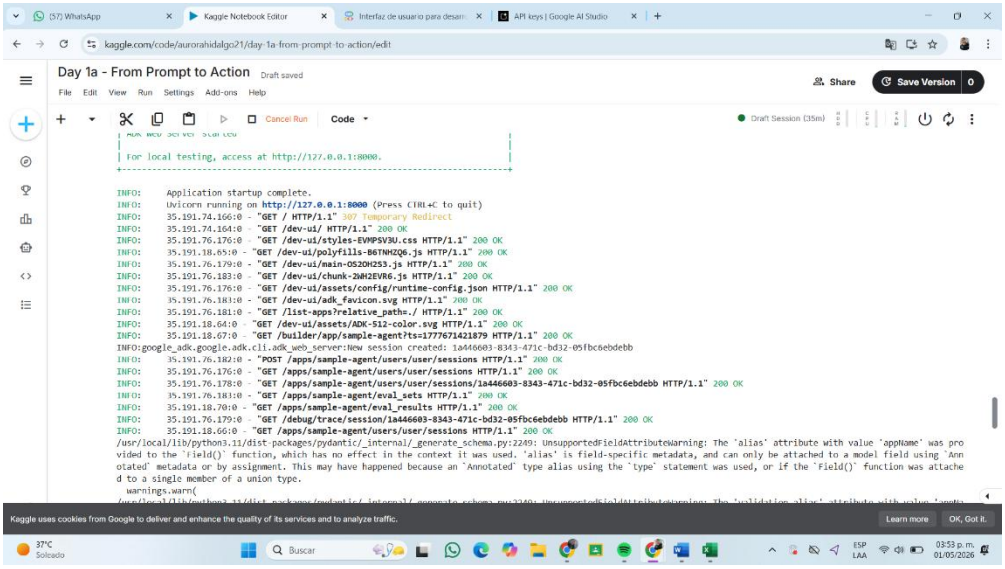


Ejecución de agentes mediante ADK.









Uso de la interfaz web ADK Web UI. Y Pruebas de interacción con el agente.

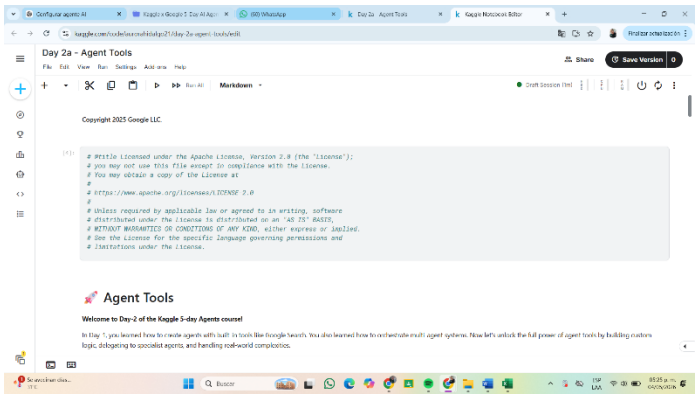
Los apredizajes obtenidos en el dia 1 fueron:

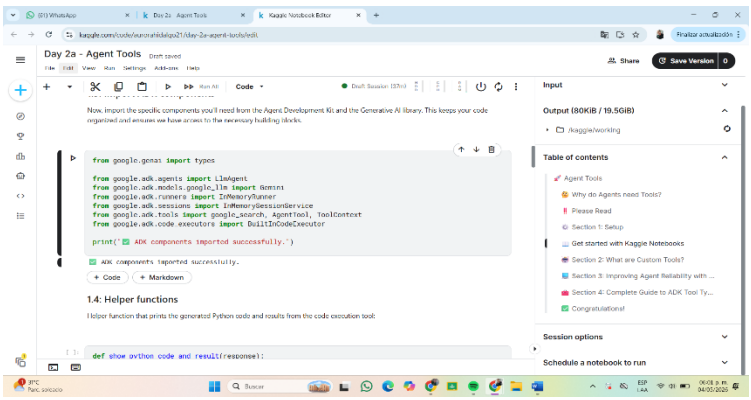
- Comprensión del funcionamiento básico de un agente de IA.
- Uso de modelos Gemini para responder consultas.
- Configuración de entornos de desarrollo para agentes.
- Diferencia entre un modelo conversacional y un agente capaz de ejecutar acciones.

El primer día permitió comprender los fundamentos del desarrollo de agentes inteligentes con Google. Además, se logró configurar correctamente el entorno de trabajo y ejecutar el primer agente funcional utilizando ADK y Gemini.

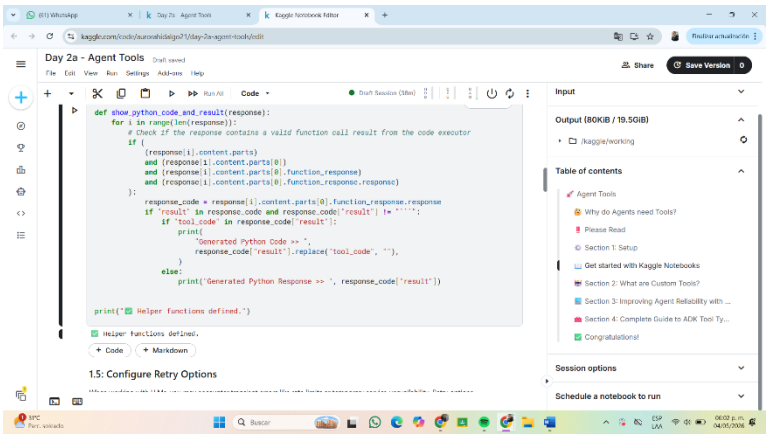
DIA 2 A – Agent Tools

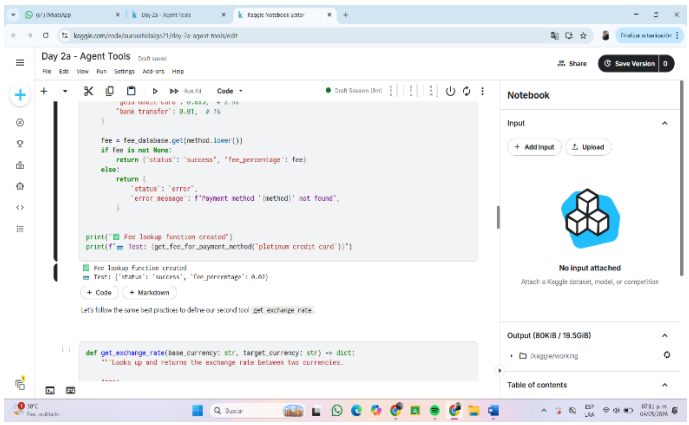
Herramientas para agentes inteligentes

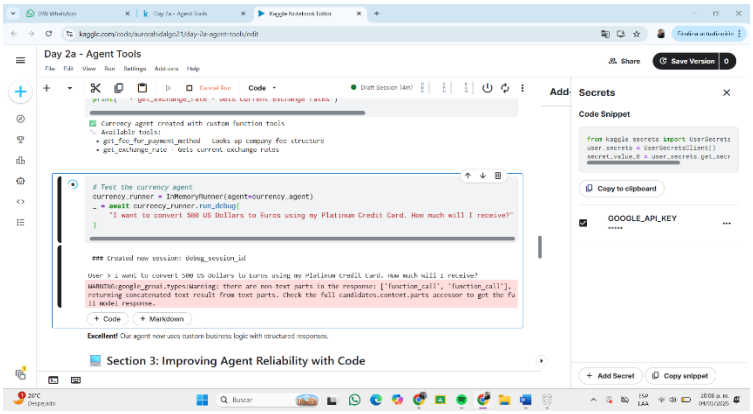


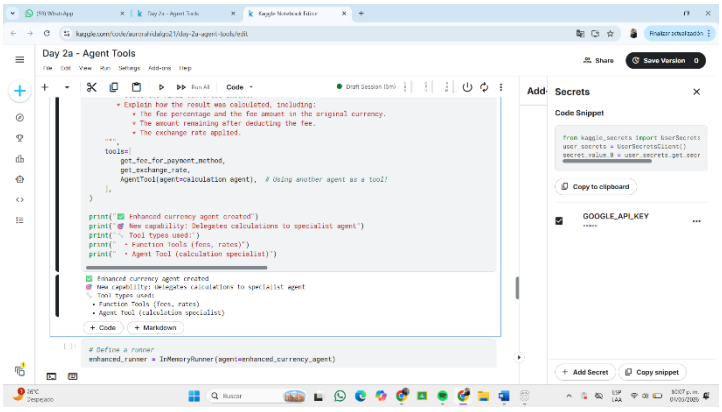


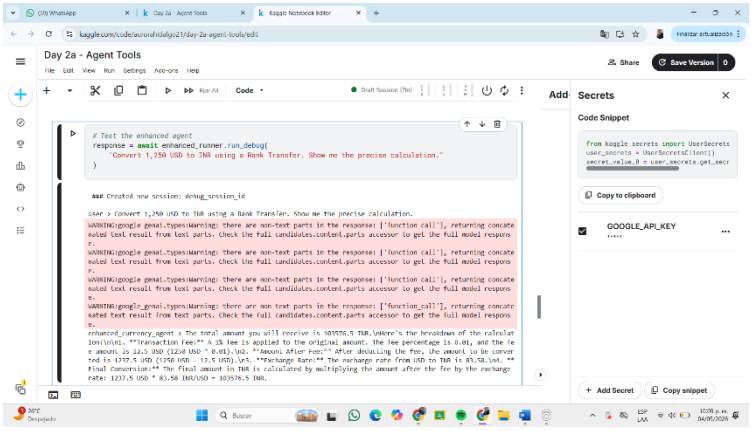
Primeramente en el dia 2A Creamosde funciones personalizadas como herramientas. Integración de herramientas dentro de agentes. Uso de múltiples tools en un solo agente.

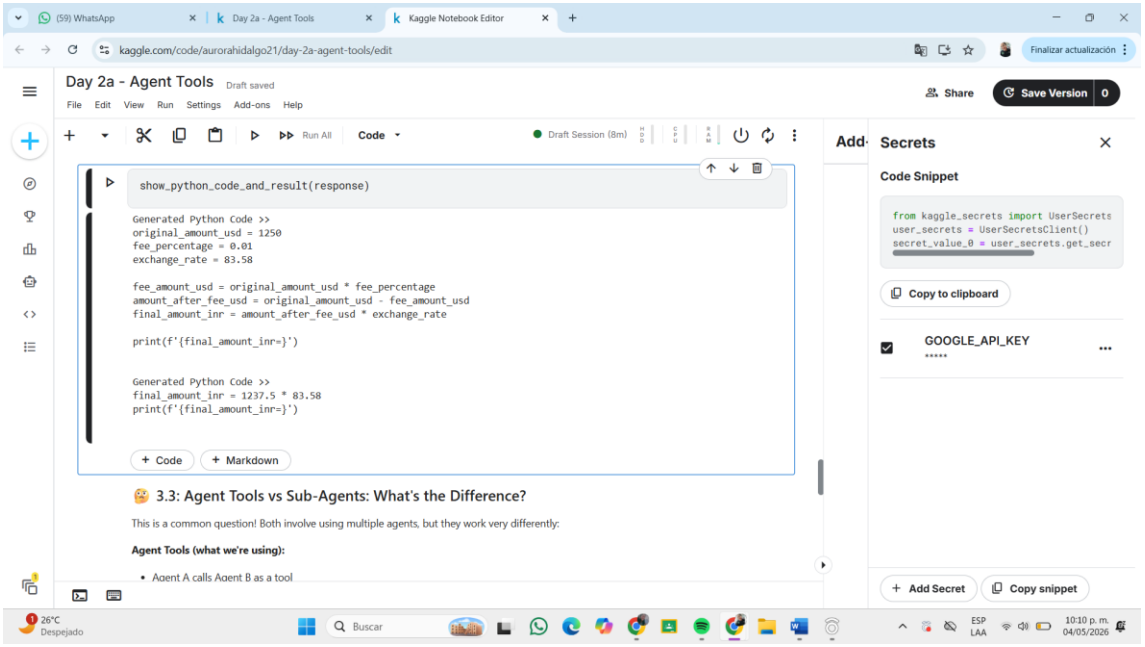












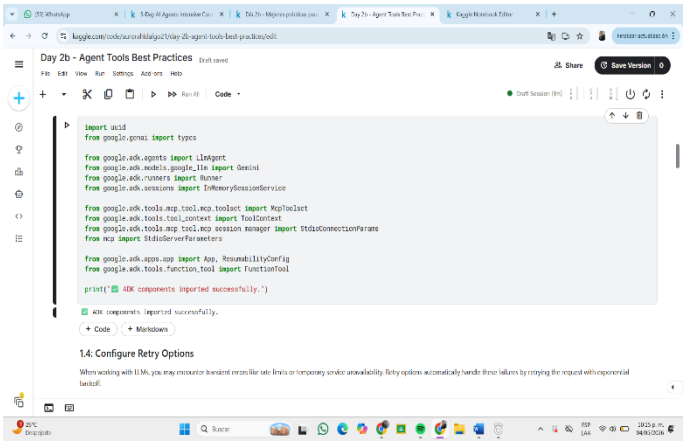
- Simulación de conversiones de divisas.
- Implementación de cálculos automáticos.
- Uso de agentes especializados.

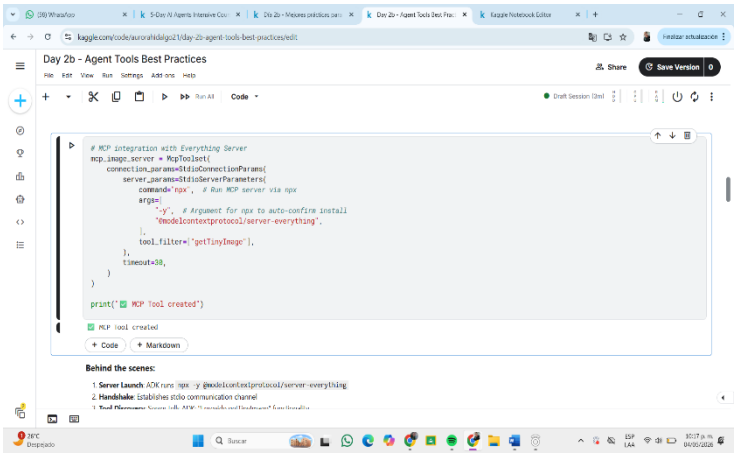
Aprendizajes obtenidos en este dia:

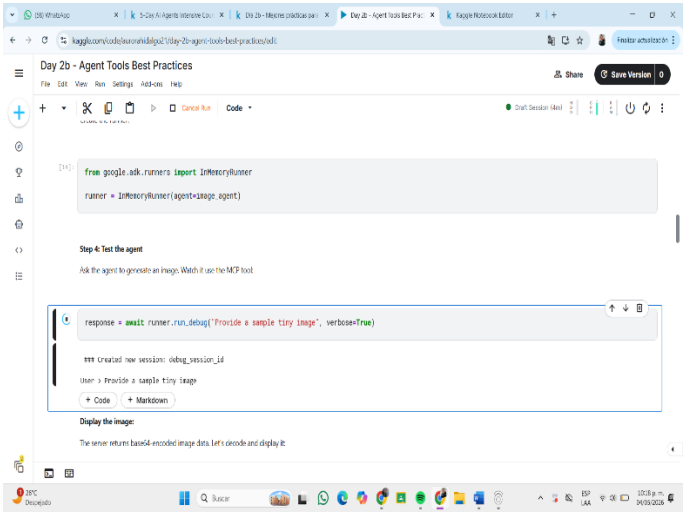
- Comprensión del concepto de Agent Tools.
- Uso de funciones Python como herramientas inteligentes.
- Integración de herramientas financieras y de conversión.
- Implementación de agentes multitarea.
- Diferencias entre herramientas y sub-agentes.

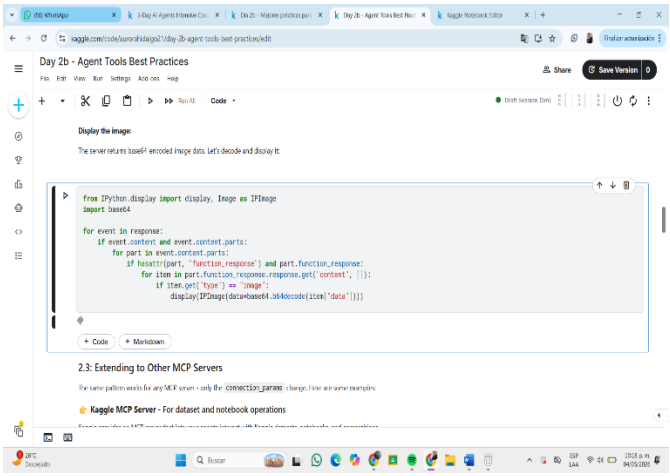
DIA 2B – Agent Tools Best Practices

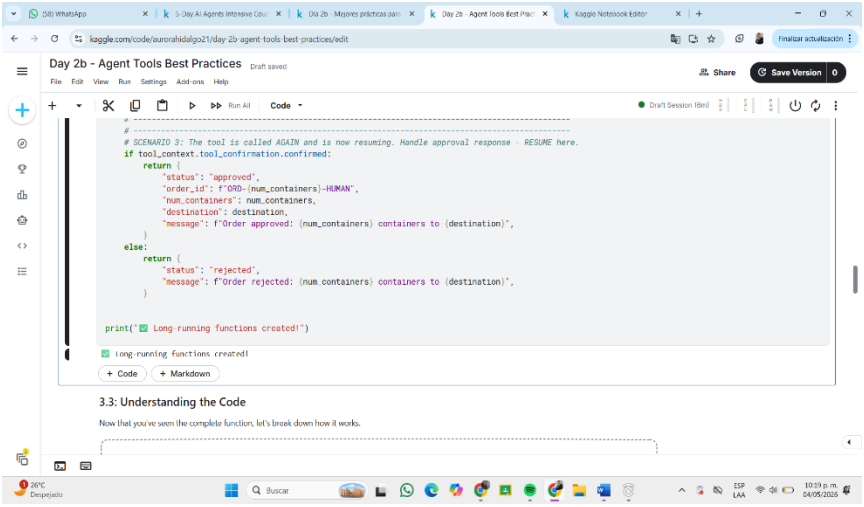
Buenas prácticas para herramientas y workflows

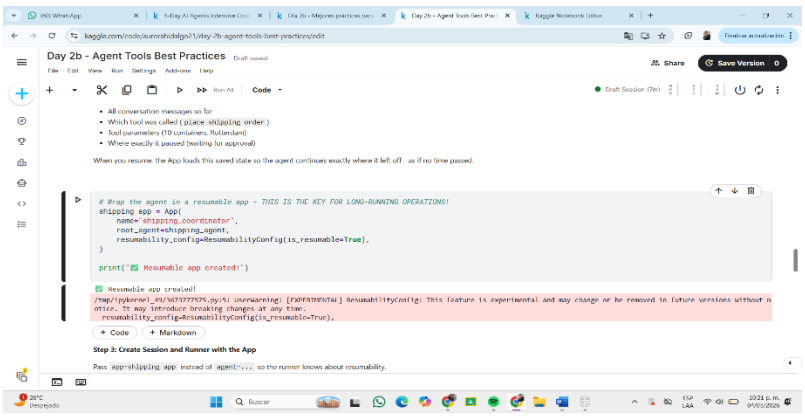


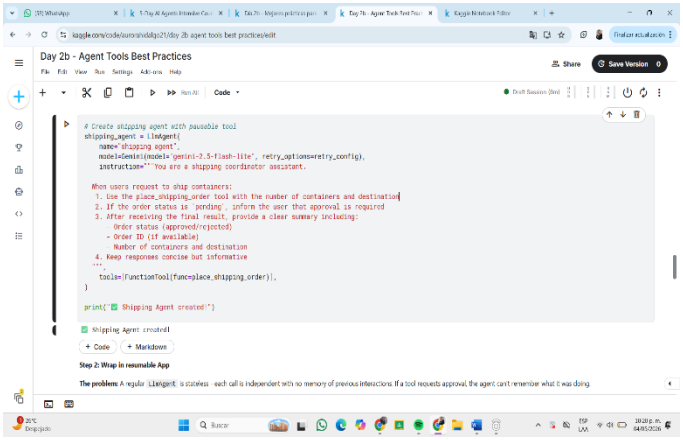


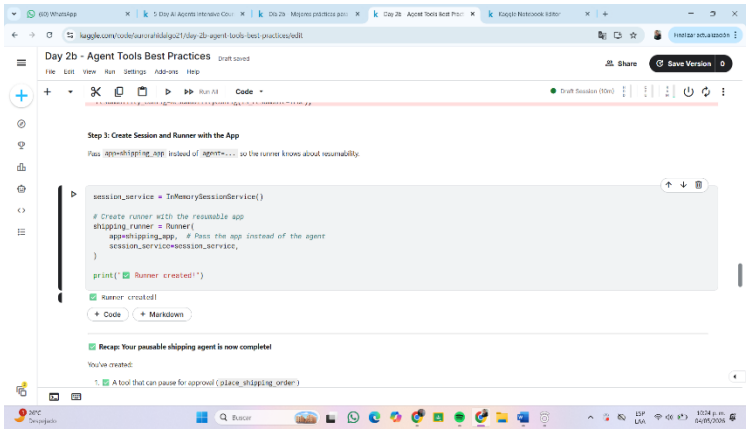


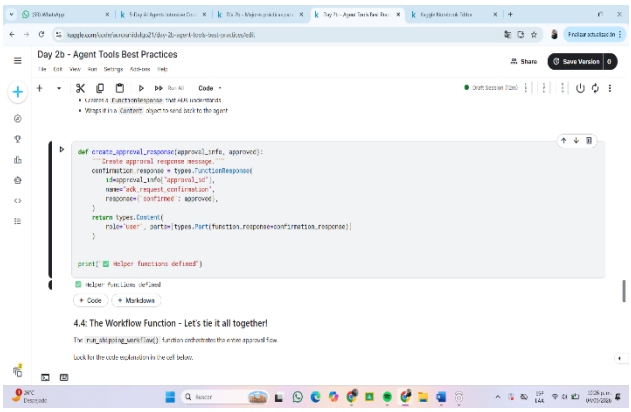


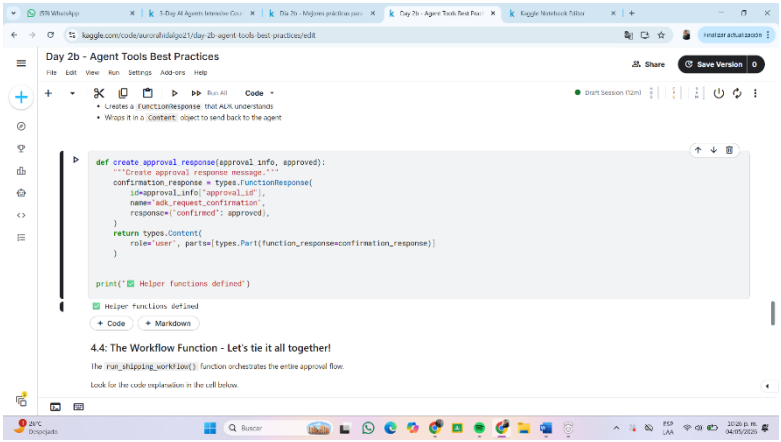


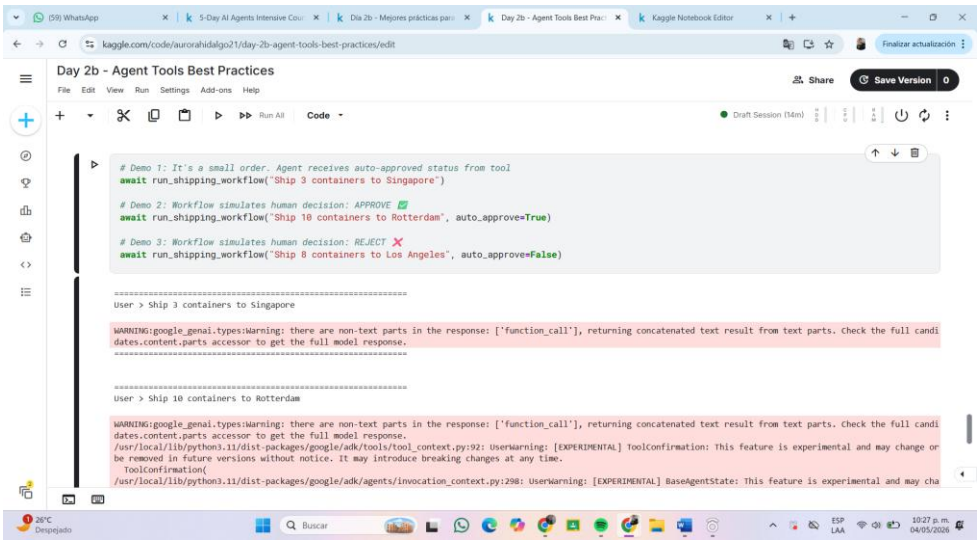

Day 2b - Agent Tools Best Practices
Share
Save Version
Code
# Wrap the agent in a resumable app - THIS IS THE KEY FOR LONG-RUNNING OPERATIONS!
shipping_app = App(
name="shipping_coordinator",
root_agent=shipping_agent,
resumability_config=ResumabilityConfig(is_resumable=True),
)
print("✅ Resumable app created!")
✅ Resumable app created!
+ Code
+ Markdown
Step 3: Create Session and Runner with the App
Pass app=shipping_app instead of agent=... so the runner knows about resumability.


Day 2b - Agent Tools Best Practices
Share
Save Version
# Create shipping agent with pausable tool
shipping_agent = LlmAgent(
name="shipping_agent",
model=Gemini(model="gemini-2.5-flash-lite", retry_options=retry_config),
instruction="""You are a shipping coordinator assistant.
When users request to ship containers:
1. Use the place_shipping_order tool with the number of containers and destination
2. If the order status is 'pending', inform the user that approval is required
3. After receiving the final result, provide a clear summary including:
- Order status (approved/rejected)
- Order ID (if available)
- Number of containers and destination
4. Keep responses concise but informative
""",
tools=[FunctionTool(func=place_shipping_order)],
)
print("✅ Shipping Agent created!")
✅ Shipping Agent created!
+ Code
+ Markdown
Step 2: Wrap in resumable App
The problem: A regular LlmAgent is stateless - each call is independent with no memory of previous interactions. If a tool requests approval, the agent can't remember what it was doing.


Day 2b - Agent Tools Best Practices
Share
Save Version
Step 3: Create Session and Runner with the App
Pass app=shipping_app instead of agent=... so the runner knows about resumability.
session_service = InMemorySessionService()
# Create runner with the resumable app
shipping_runner = Runner(
app=shipping_app, # Pass the app instead of the agent
session_service=session_service,
)
print("✅ Runner created!")
✅ Runner created!
+ Code
+ Markdown
Recap: Your pausable shipping agent is now complete!
You've created:
1. A tool that can pause for approval (place_shipping_order)


Day 2b - Agent Tools Best Practices
Share
Save Version
def create_approval_response(approval_info, approved):
"""Create approval response message."""
confirmation_response = types.FunctionResponse(
id=approval_info['approval_id'],
name='adk_request_confirmation',
response={'confirmed': approved},
)
return types.Content(
role='user', parts=[types.Part(function_response=confirmation_response)]
)
print('✅ Helper functions defined')
✅ Helper functions defined
+ Code
+ Markdown
4.4: The Workflow Function - Let's tie it all together!
The run_shipping_workflow() function orchestrates the entire approval flow.
Look for the code explanation in the cell below.


Day 2b - Agent Tools Best Practices
Share
Save Version
Creates a functionResponse that ADK understands
Wraps it in a Content object to send back to the agent
def create_approval_response(approval_info, approved):
"""Create approval response message."""
confirmation_response = types.FunctionResponse(
id=approval_info['approval_id'],
name='adk_request_confirmation',
response={'confirmed': approved},
)
return types.Content(
role='user', parts=[types.Part(function_response=confirmation_response)]
)
print('✅ Helper functions defined')
✅ Helper functions defined
+ Code
+ Markdown
4.4: The Workflow Function - Let's tie it all together!
The run_shipping_workflow() function orchestrates the entire approval flow.
Look for the code explanation in the cell below.



Se trabajó principalmente con:

- Workflows de aprobación.
- Operaciones de larga duración.
- Confirmaciones humanas.
- Manejo de estado.
- Reanudación de proces

ACTIVIDADEDS REALISADAS EN ESTE DIA

- Simulación de órdenes de envío.
- Implementación de aprobaciones automáticas.
- Uso de confirmación humana para operaciones críticas.
- Pausa y reanudación de workflows.
- Uso de ToolContext y estados persistentes.

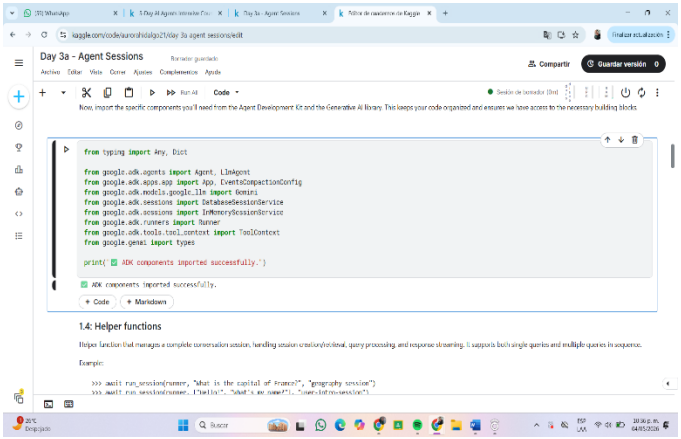
APRENDIZAJE OBTENIDO

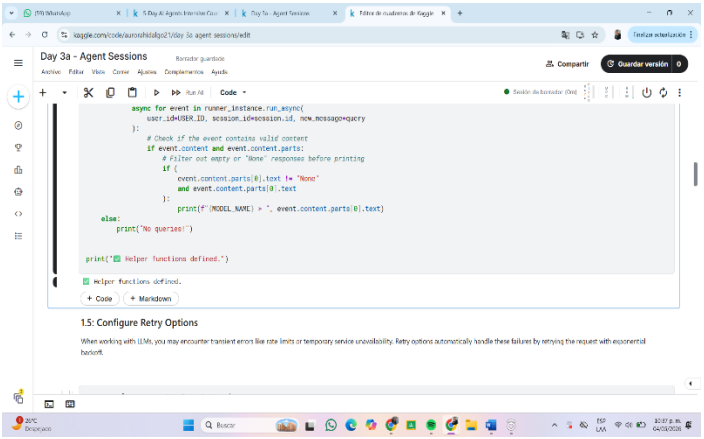
- mportancia de las aprobaciones humanas en sistemas inteligentes.
- Manejo de operaciones complejas en agentes.
- Implementación de workflows seguros.
- Reanudación de procesos mediante invocation_id.
- Uso de patrones avanzados de automatización.

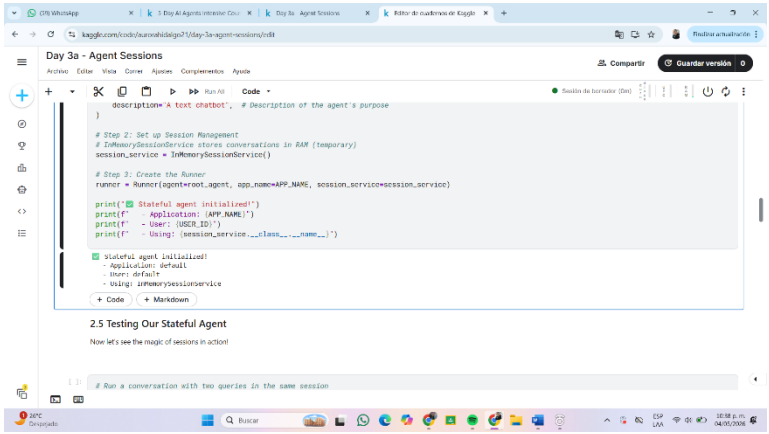
DIA 3 Gestión de estado y memoria

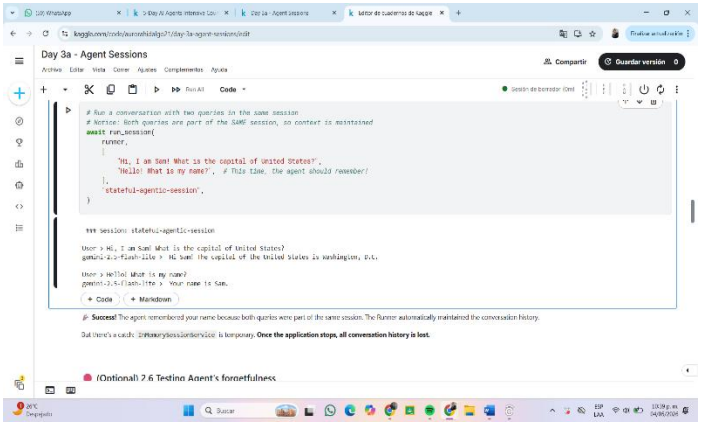
Durante el Día 3 se introdujeron conceptos relacionados con el manejo de estado y memoria dentro de los agentes inteligentes.

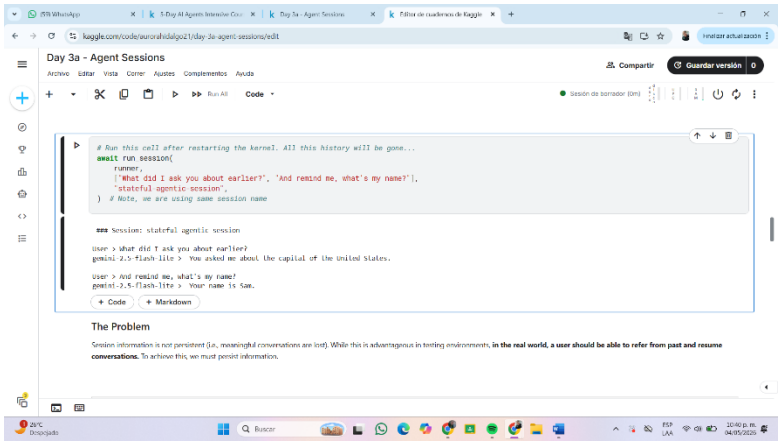
Se analizó cómo los agentes pueden recordar información de conversaciones anteriores y mantener contexto entre múltiples interacciones.

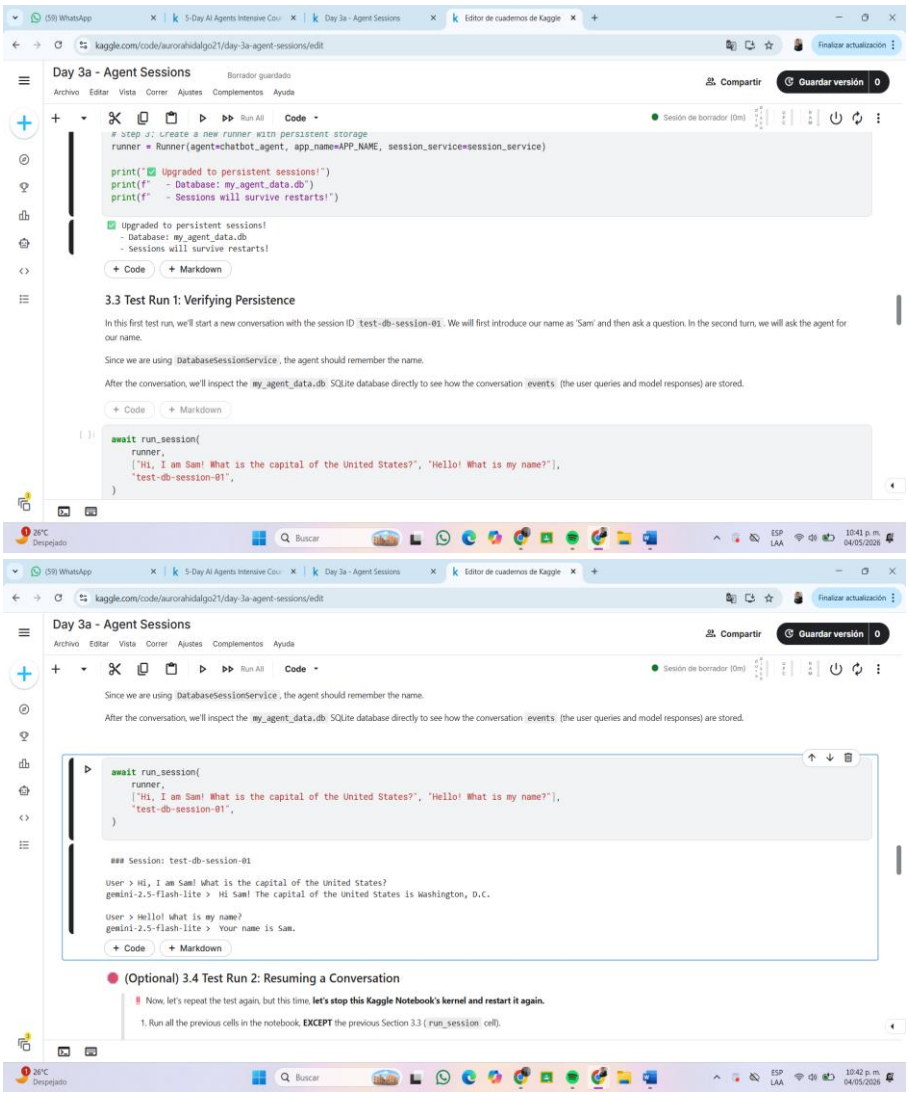












Aprendizajes obtenidos

- Importancia del manejo de memoria en agentes.
- Diferencia entre sesiones independientes.
- Conservación del contexto conversacional.
- Persistencia de datos entre ejecuciones.